

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

TN SKILL PROJECT DOCUMENTATION

Academic Year 2026

Submitted By

Name: MUTHU SELVI P

Register Number: 24083021802112121

Department: BSC COMPUTER SCIENCE

College: ARIGNAR ANNA COLLEGE

Team Members

1. MAHESHWARI R
2. MADHU VARSHA S
3. MUTHU LASHMI M
4. LAKSHMANAN K

ABSTRACT

The project “**Implement Client Script & UI Policy (Incident)**” focuses on improving data accuracy, consistency, and user interaction in the ServiceNow Incident Management system. Incident records require complete and accurate information for effective triage, routing, reporting, and resolution. Manual data entry may result in incomplete, inconsistent, or incorrect information, which can affect reporting accuracy, SLA compliance, and overall service quality.

This project implements **UI Policies, UI Policy Actions, and Client Scripts** to control the behaviour of Incident form fields based on specific conditions. When an Incident has High Impact, the system applies additional controls to the Incident form. The Assignment Group can be made mandatory, and the Urgency field can be controlled through a UI Policy Action.

An **onChange Client Script** is implemented to automatically set the Urgency to High when the Impact is High. An **onSubmit Client Script** validates the Assigned To field before allowing the Incident record to be saved. An **onCellEdit Client Script** prevents users from changing the State field directly through list editing.

The implementation is tested using different scenarios, including mandatory enforcement, successful save, reverse condition testing, list-edit blocking, and form-based updates. Overall, the project demonstrates how ServiceNow client-side controls can enforce business rules and improve data quality and consistency in Incident Management.

1. INTRODUCTION

ServiceNow is a platform used to manage IT services and business processes. Incident Management is an important part of ServiceNow that helps organizations record, track, assign, and resolve incidents.

Incident records contain important information such as Impact, Urgency, Assignment Group, Assigned To, and State. For effective Incident Management, this information needs to be complete, accurate, and consistent.

Relying only on users to manually enter correct information may result in incomplete or incorrect records. Such issues can affect incident routing, reporting accuracy, SLA compliance, and service quality.

To address this problem, ServiceNow provides client-side configuration features such as **UI Policies and Client Scripts**. These features can dynamically control field behavior, make fields mandatory, automatically populate values, validate information, and prevent invalid actions.

This project demonstrates the implementation of these controls on the ServiceNow Incident table.

The main focus is to apply appropriate rules when the Incident **Impact** is set to High and to validate user actions before information is submitted or modified.

2. PROBLEM STATEMENT

Incident records require consistent and accurate data entry for effective triage, routing, and resolution.

However, relying solely on user awareness and manual checks can lead to:

- Incomplete Incident records.
- Incorrect or inconsistent data.
- Missing mandatory information.
- Improper field values.
- Unauthorized changes through list editing.
- Reduced reporting accuracy.
- Problems with SLA compliance.
- Reduced overall service quality.

To overcome these problems, conditional field behavior and validation need to be enforced directly at the user-interface level.

Therefore, this project implements standardized **UI Policies and Client Scripts** to control field visibility, mandatory status, field behavior, automatic updates, and save-time validation.

The objective is to ensure that Incident records meet the defined business rules before they are submitted.

3. OBJECTIVES

The main objective of this project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records.

Specific Objectives

1. To create a UI Policy for High Impact Incidents.
2. To make the Assignment Group field mandatory when Impact is High.
3. To control the Urgency field when Impact is High.
4. To automatically set Urgency to High using an onChange Client Script.
5. To prevent saving a High Impact Incident when Assigned To is empty.
6. To prevent direct State changes through Incident list editing.
7. To allow valid State changes through the Incident form.
8. To test the configured UI Policies and Client Scripts.
9. To ensure that the configured rules work correctly when conditions change.
10. To improve data quality and consistency in Incident Management.

The project is designed as a practical demonstration of Incident Management, UI Policy, UI Policy Actions, Client Scripts, and Form Validation.

4. SKILLS AND CONCEPTS

The following ServiceNow concepts are used in this project:

4.1 Incident Management

Incident Management is used to create, manage, assign, and update Incident records.

4.2 UI Policy

A UI Policy dynamically controls the behavior of fields on a form based on a defined condition.

4.3 UI Policy Action

A UI Policy Action defines the specific behavior of an individual field when the UI Policy condition is satisfied.

4.4 Client Script

Client Scripts are used to perform client-side actions and validations on ServiceNow forms.

4.5 Form Validation

Form validation ensures that required information is available before an Incident is submitted.

4.6 JavaScript

JavaScript is used to implement the required Client Script logic.

The source project specifically identifies Incident Management, UI Policy, UI Policy Actions, Client Scripts, and Form Validation as the main skills demonstrated.

5. SCOPE OF THE PROJECT

The scope of this project is focused on controlling and validating Incident records in ServiceNow.

The project includes:

- Creating a UI Policy for High Impact Incidents.
- Making Assignment Group mandatory.
- Controlling the Urgency field.
- Automatically setting Urgency based on Impact.
- Validating Assigned To before saving.
- Preventing State changes through direct list editing.
- Allowing State changes through the Incident form.
- Testing the configured behaviors.

Fields Used

The main Incident fields involved are:

- Impact
- Assignment Group
- Assigned To
- Urgency
- State

The implementation is limited to the Incident table and its client-side behavior.

6.TECHNOLOGIES USED

Technology / Component	Purpose
ServiceNow	Development and implementation platform
Incident Management	Module used for the project
UI Policy	Controls dynamic field behavior
UI Policy Action	Controls individual field properties
Client Script	Performs automation and validation
JavaScript	Used to write Client Scripts
Web Browser	Used to access ServiceNow

7. SYSTEM REQUIREMENTS

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Stable Internet Connection

Software Requirements

- Web Browser
- ServiceNow Instance
- ServiceNow Incident Management
- Administrative or configuration access

Administrative or configuration access is required for creating and configuring the UI Policies and Client Scripts used in the project.

8. PROJECT MODULES

The project consists of the following modules:

Module 1 – High Impact UI Policy

Creates a UI Policy that is triggered when the Incident Impact is High.

Module 2 – Assignment Group UI Policy Action

Makes Assignment Group mandatory when the UI Policy condition is satisfied.

Module 3 – Urgency UI Policy Action

Makes the Urgency field read-only when Impact is High.

Module 4 – onChange Client Script

Automatically sets Urgency to High when Impact changes to High.

Module 5 – onSubmit Client Script

Prevents the Incident from being saved when Assigned To is empty under the specified condition.

Module 6 – onCellEdit Client Script

Prevents direct State changes from the Incident list.

Module 7 – Testing

Tests all implemented configurations under different conditions.

9. TASK 1 – CREATE UI POLICY ON INCIDENT

9.1 Create UI Policy

1. The first step is to create a UI Policy named:

- High Impact Control
- Navigation
- System UI → UI Policies

2. Click **New** to create the UI Policy.

3. Configuration

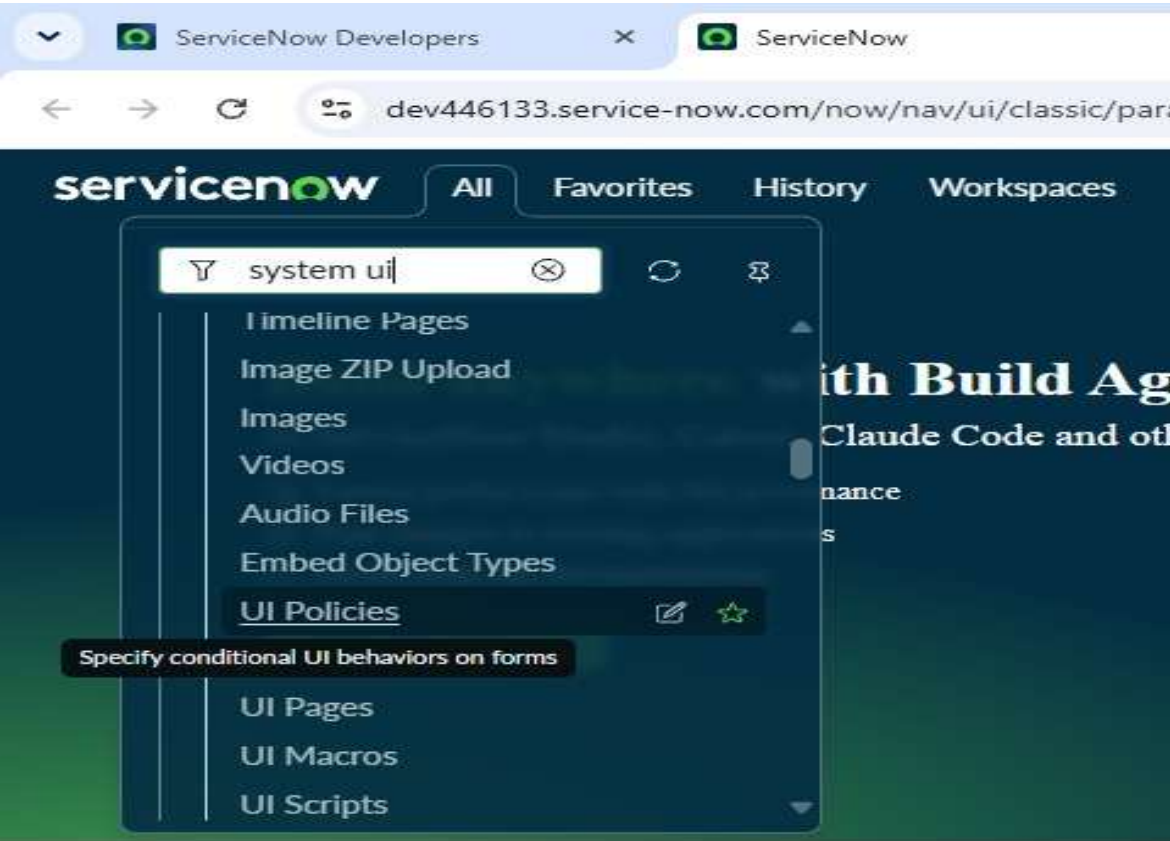
Property	Value
Name	High Impact Control
Table	Incident
Active	True
Condition Field	Impact
Operator	is
Value	1 – High
Reverse if false	True

Condition

The UI Policy is triggered when:

Impact = 1 – High

This condition ensures that the UI Policy is activated only for High Impact Incidents. The **Reverse if false** option ensures that the changes applied by the UI Policy are reversed when the condition is no longer satisfied.



The screenshot shows the ServiceNow UI Policies list page. The browser address bar displays the URL: `dev446133.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy_list.do`. The ServiceNow logo is visible at the top left, followed by navigation tabs: All, Favorites, History, and Workspaces. The "UI Policies" tab is selected. A search bar and a "Search" button are present. Below the search bar, a table lists the UI Policies. The table has columns: Short description, Table, Conditions, Reverse if false, On load, Updated, and Order. The table contains 8 rows of data.

Short description	Table	Conditions	Reverse if false	On load	Updated	Order
Make assignment group mandatory	Task [task]		true	true	2026-06-12 14:20:21	100
Handle use conditions flag	Workflow Version [wf_workflow_version]	condition_type!=^EQ	true	true	2026-06-12 14:20:21	100
Show pricing implications checkbox	Question [question]	type=8^ORtype=21^EQ	true	true	2026-06-12 14:19:23	100
Show price if checked	Question [question]	type=7^EQ	true	true	2026-06-12 14:19:23	100
Duration only applies to request type categories	Dynamic category [sc_category_top_n]	type=sc_req_item^EQ	true	true	2026-06-12 14:19:23	100
Show Multiple Choices and Choice price	Item Option [sc_cat_item_option]	type=multiple_choice^ORtype=numeric_scal...	true	true	2026-06-12 14:19:23	100
Hide Source	Catalog Client Scripts [catalog_script_client]	applies_to=item^EQ	true	true	2026-06-12 14:19:23	100
Show Macro	Question [question]	type=14^ORtype=17^EQ	true	true	2026-06-12 14:18:29	100

ServiceNow Developers

New Record | UI Policy | ServiceNow

Ask Gemini

dev446133.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy.do%3Fsys_id%3D-1%26sys_is_list%3Dtrue%26sys_target%3Dsys_ui_policy%2...

Verify it's you

serviceNow

All Favorites History Workspaces Admin

UI Policy - New Record

Search

UI Policy

New record

View: Advanced*

Submit

More info

Table Incident [Incident]

Application Global

Active

* Short description High Impact Control

Order 100

When to Apply

Script

Conditions

Add Filter Condition

Add OR Clause

Impact

is

1 - High

AND

OR

If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific

Apply the UI policy actions when the form is loaded and when the user changes values on the form

Global

On load

Reverse the effects of the UI policy actions when the Conditions evaluate to false

Tables that extend the specified Table inherit this UI Policy

Reverse if false

Inherit

Submit

Related Links

Default view

29°C Mostly cloudy

Search

ENG IN

68%

12:36 25-09-2026

10. UI POLICY ACTION – ASSIGNMENT GROUP

After creating the **High Impact Control** UI Policy, a UI Policy Action is created for the Assignment Group field.

Configuration

Property	Value
Field	Assignment Group
Mandatory	True

Function

When the Incident Impact is set to High:

Assignment Group → Mandatory

This ensures that the Incident contains appropriate assignment information before it is submitted.

When the Impact is changed to a value that does not satisfy the UI Policy condition, the field behavior is reversed because **Reverse if false** is enabled.

11. UI POLICY ACTION – URGENCY

A second UI Policy Action is created for the Urgency field.

Configuration

Property	Value
Field Name	Urgency
Read-only	True
Visible	Leave unchanged

Function

When:

Impact = High

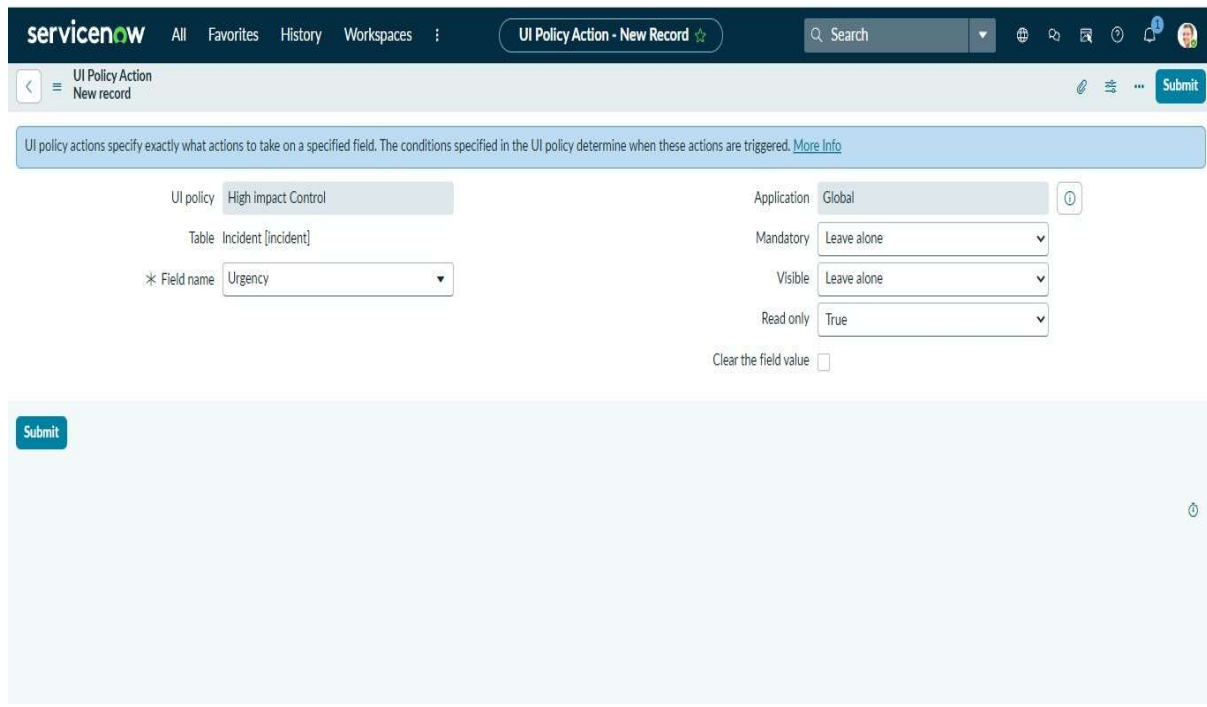
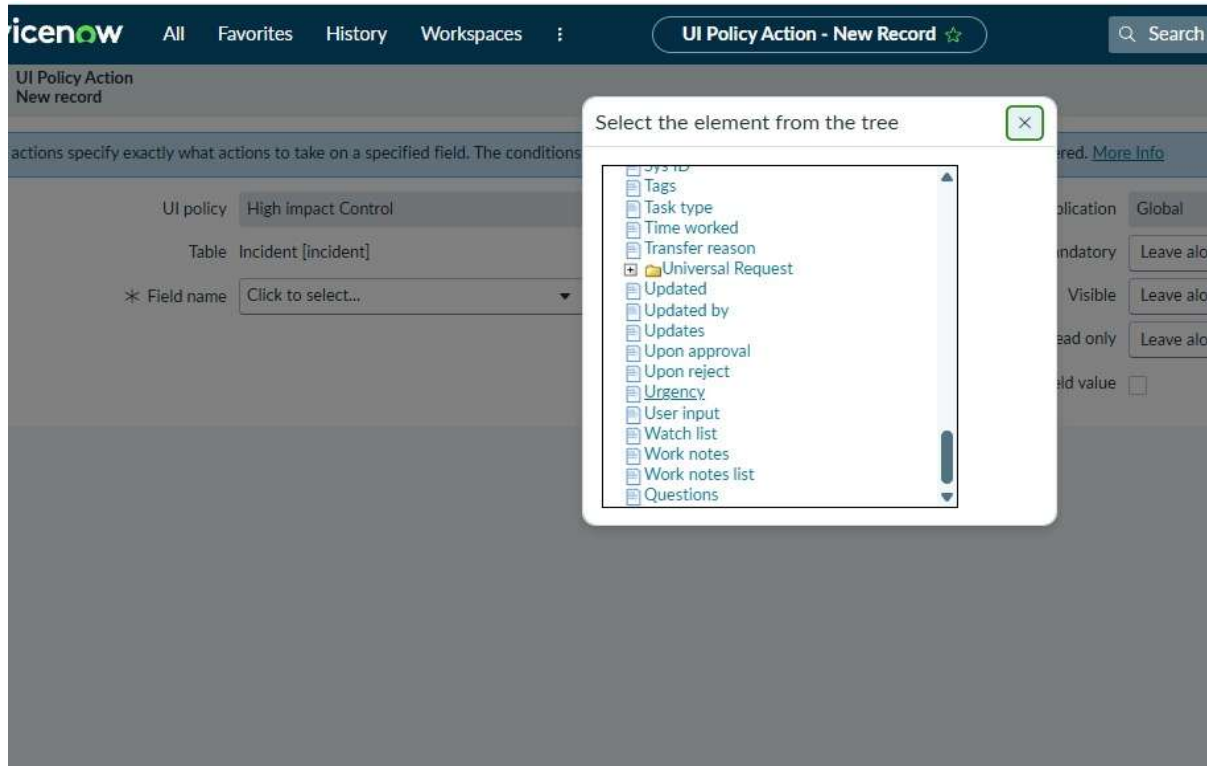
the Urgency field becomes **read-only**.

The field remains visible on the Incident form, but users cannot manually change its value while the UI Policy condition is active.

Verification

1. Open an Incident record.
2. Set Impact to High.
3. Observe the Urgency field.
4. Verify that Urgency becomes read-only.
5. Change Impact to another value.
6. Verify that the field becomes editable again.

The source document specifies this UI Policy Action and its verification procedure.



12. TASK 3 – CREATE ONCHANGE CLIENT SCRIPT

Script Name:

Auto set urgency for high impact

Configuration

Property	Value
Name	Auto set urgency for high impact
Table	Incident
Type	onChange
Field Name	Impact
Active	True

Purpose

The script runs whenever the Impact field changes.

If Impact is set to High, the script automatically sets Urgency to High.

It also displays an informational message to the user.

Script

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == "") {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage(  
            'Urgency set to High for High impact incident.'  
        );  
    }  
}
```

Expected Behavior

When: **Impact = High**

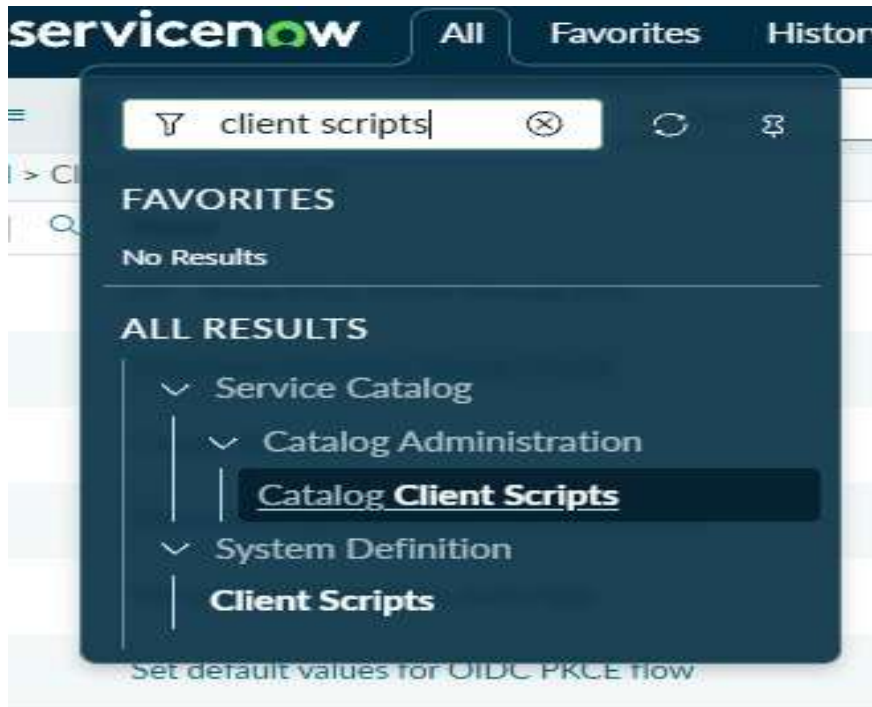
the system automatically performs:

Urgency = High

and displays:

“Urgency set to High for High impact incident.”

The source document specifies the onChange Client Script, its configuration, and script behavior.



servicenow All Favorites History Workspaces Admin Client Script - Auto set urgency for high impact Search

Client Script Auto set urgency for high impact Update Delete

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name	Auto set urgency for high impact	Application	Global
Table	Incident [incident]	Active	☒
UI Type	Desktop	Inherited	☐
Type	onChange	Global	☒
Field name	Impact		
Description			
Messages			
Script	<pre>1 function onChange(control, oldValue, newValue, isLoading) { 2 if(!isLoading newValue === '') { 3 return; 4 } 5 if(newValue=='1'){ 6 _form.setValue('urgency','1'); 7 _form.addInfoMessage('Urgency set to High for High impact incident.');8 } 9 }</pre>		
Isolate script	☒		

13. TASK 4 – CREATE ONSUBMIT CLIENT SCRIPT

Script Name:

Prevent save if Assigned To missing

Configuration

Property	Value
Name	Prevent save if Assigned To missing
Table	Incident
Type	onSubmit
Active	True

Purpose

This Client Script validates the Incident when the user attempts to save it.

The script checks whether:

Impact = High

and

Assigned To = Empty

If both conditions are true, the Incident record is not saved.

Script

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == "") {  
        g_form.showErrorBox(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
    return true;  
}
```

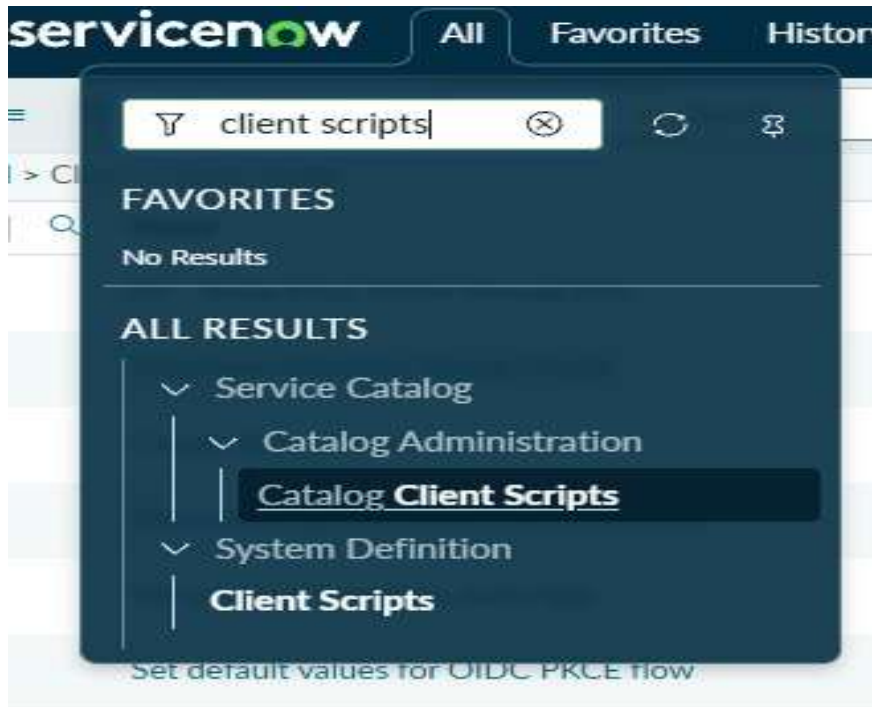
Expected Behavior

If a High Impact Incident does not have an Assigned To user:

- Save operation is stopped.
- Error message is displayed.
- Incident is not submitted.

If Assigned To is filled, the Incident can be saved.

The source document specifies this validation and testing behavior.



ServiceNow Client Script - Prevent save if Assigned To missing

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name	Prevent save if Assigned To missing	Application	Global
Table	Incident [Incident]	Active	☒
UI Type	Desktop	Inherited	☐
Type	onSubmit	Global	☒

Description

Messages

Script

```
1 function onSubmit() {  
2   if(g_form.getValue('impact')==1&&  
3     g_form.getValue('assigned_to')==){  
4     g_form.showErrorBox(  
5       'assigned_to',  
6       'Assigned To is mandatory for High Impact incidents,'  
7     );  
8     return false;  
9   }  
10  return true;  
11 }
```

Isolate script ☒

14. TASK 5 – CREATE ONCELLEDIT CLIENT SCRIPT

Script Name:

Prevent state change via list edit

Configuration

Property	Value
Name	Prevent state change via list edit
Table	Incident
Type	onCellEdit
Field Name	State
Active	True

Purpose

This Client Script prevents users from changing the State field directly through Incident list editing.

Script

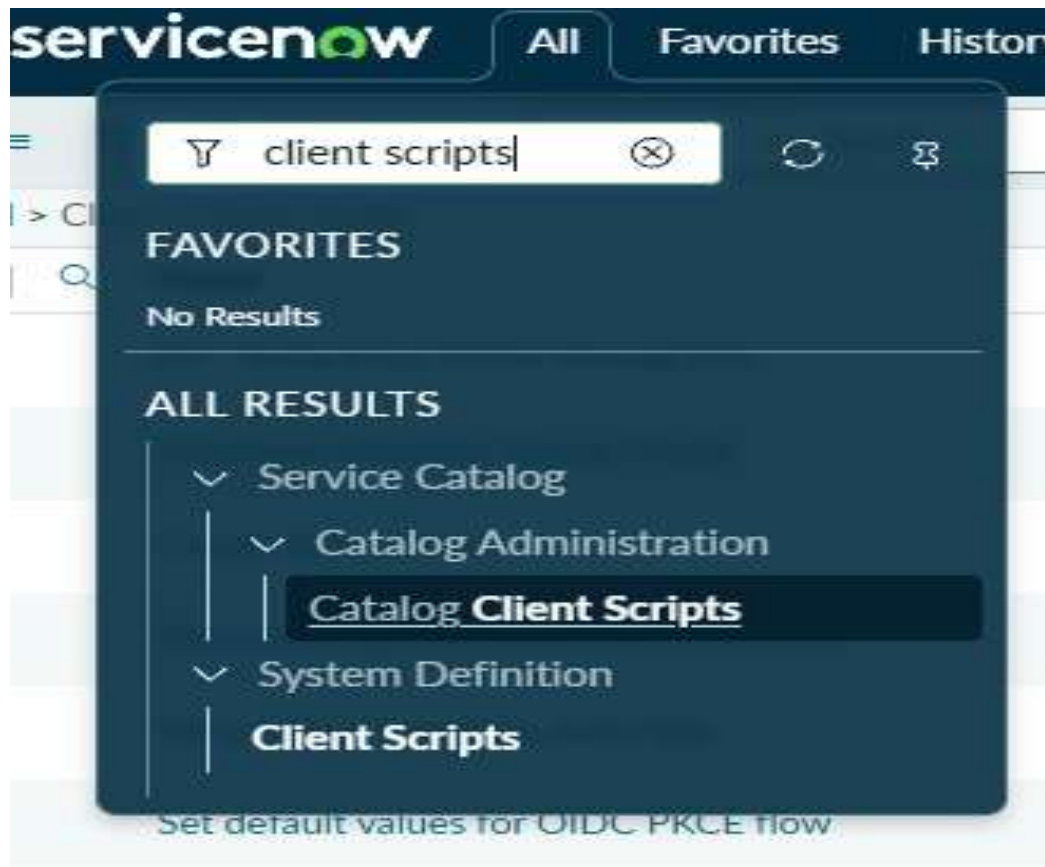
```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert(  
        'State cannot be updated using list editing. Please open the Incident.'  
    );  
    callback(false);  
}
```

Expected Behavior

When a user attempts to edit the State directly from the Incident list:

1. Alert message is displayed.
2. The list edit operation is blocked.
3. The State value remains unchanged.

The source document defines this Client Script specifically for preventing State changes through list editing.



The screenshot shows the ServiceNow interface for configuring a Client Script. The title bar indicates the script is named 'Prevent state change via list edit'. A blue informational banner at the top states: 'New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.'

The configuration form includes the following fields:

- Name:** Prevent state change via list edit
- Table:** Incident [Incident]
- UI Type:** Desktop
- Type:** onCellEdit
- Field name:** State
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒
- Description:** (empty text area)
- Messages:** (empty text area)
- Script:**

```
1 function onCellEdit(sysId, table, oldvalues, newvalue, callback) {  
2  
3   alert('State cannot be updated using list editing. Please open the Incident.');
```
- Isolate script:** ☒

At the bottom of the form are 'Update' and 'Delete' buttons.

15. PROJECT WORKFLOW

The overall working flow of the project is:

START



Open / Create Incident



Select Impact



Is Impact High?



YES



High Impact Control UI Policy



Assignment Group becomes Mandatory



Urgency becomes Read-only



onChange Client Script



Urgency automatically set to High



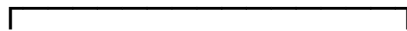
User clicks Submit



onSubmit Client Script



Is Assigned To filled?



NO



Block Save



YES



Save Incident



Display Error

List Editing Flow

Incident List



User edits State



onCellEdit Client Script



Display Alert



callback(false)



State Change Blocked

Form Update Flow

Open Incident Form



Change State



Click Update



State Change Saved

16. TESTING

Testing is performed to verify that every UI Policy and Client Script works according to the defined requirements.

Test Case 1 – Mandatory Enforcement

Steps:

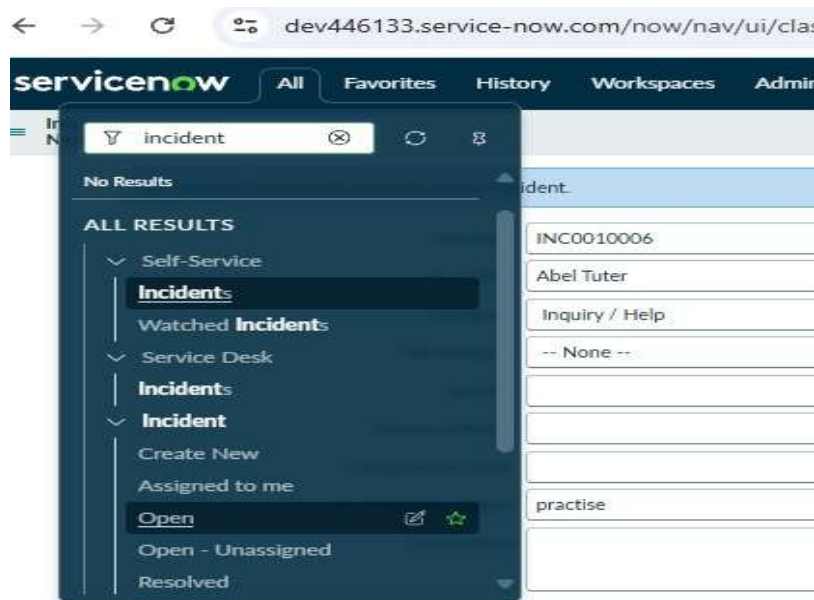
1. Navigate to Incident → Create New.
2. Set Impact to High.
3. Leave Assigned To empty.
4. Click Submit.

Expected Result:

The Incident should not be saved.

An error message should indicate that Assigned To is mandatory for High Impact Incidents.

This test verifies the onSubmit validation and mandatory enforcement.



Urgency set to High for High Impact incident.

Number	INC0010005	Channel	-- None --
* Caller	Abel Tuter	State	New
Category	Inquiry / Help	Impact	1 - High
Subcategory	-- None --	Urgency	1 - High
Service		Priority	1 - Critical
Service offering		* Assignment group	CAB Approval
Configuration item		Assigned to	
* Short description	practise		
Description			

Related Search Results

Related Search practise Knowledge & Catalog (All)

No matching results found for practise

NotesRelated RecordsResolution Information

Watch list

Work notes list

Test Case 2 – Successful Save

Steps:

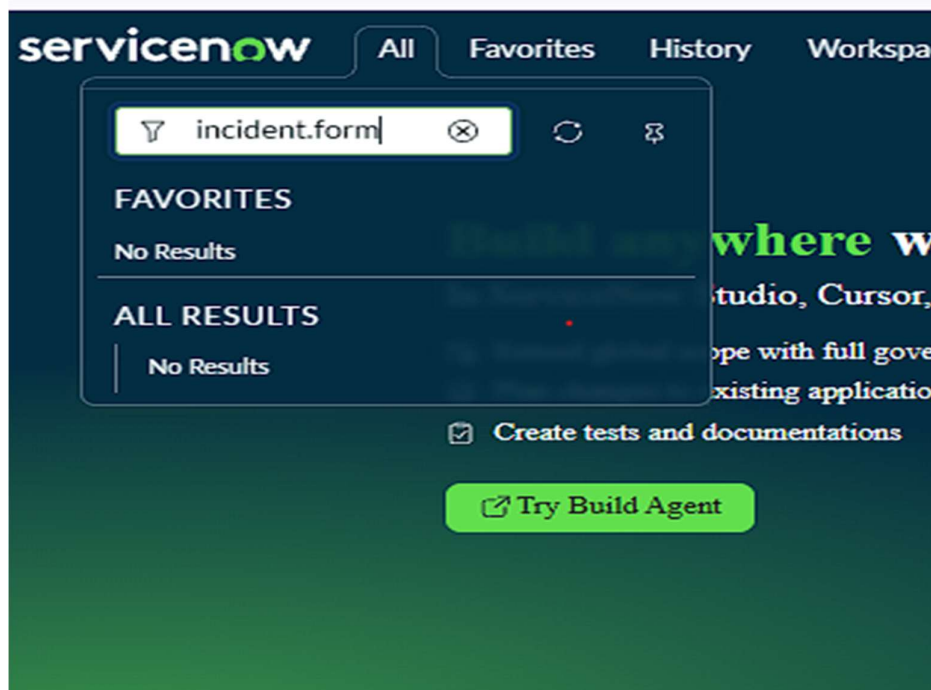
1. Open the Incident form.
2. Set Impact to High.
3. Fill the Assigned To field.
4. Click Submit.

Expected Result:

The Incident should be saved successfully.

The following behaviors should also be verified:

- Urgency auto-setting.
- Read-only field behavior.
- State restrictions.



servicenow

AllFavoritesHistoryWorkspacesAdmin

Incident - Create INC0010005

Search

Incident

New record

Submit

Resolve

Urgency set to High for High impact incident.

Number

INC0010005

* Caller

Abel Tuter

Category

Inquiry / Help

Subcategory

-- None --

Service

Service offering

Configuration item

* Short description

Description

Channel

-- None --

State

New

Impact

1 - High

Urgency

1 - High

Priority

1 - Critical

Assignment group

CAB Approval

Assigned to

cab approver

Related Search Results

Related Search

Knowledge & Catalog (All)

No results to display

17. REVERSE CONDITION TEST

Test Case 3 – Reverse Condition

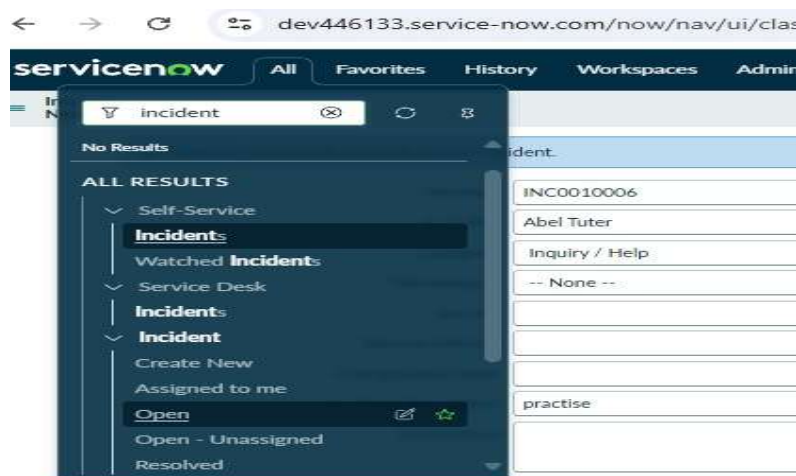
Steps:

1. Open an existing Incident where Impact is High.
2. Change Impact from High to Medium.
3. Observe Assignment Group.
4. Observe Urgency.
5. Save the Incident.

Expected Result

- Assignment Group should no longer be mandatory.
- Urgency should become editable.
- UI Policy changes should be reversed.
- The record should save successfully.

This verifies the **Reverse if false** functionality of the UI Policy.



The screenshot shows the ServiceNow 'Incident - Create INC0010008' form. The form is divided into two main sections: a top section for incident details and a bottom section for related search results.

Incident Details Section:

- Number:** INC0010008
- Channel:** -- None --
- * Caller:** Abel Tuter
- State:** New
- Category:** Inquiry / Help
- Impact:** 2 - Medium
- Subcategory:** -- None --
- Urgency:** 3 - Low
- Service:** (empty)
- Priority:** 4 - Low (with a 'Lookup using list' button)
- Service offering:** (empty)
- Assignment group:** CAB Approval
- Configuration item:** (empty)
- Assigned to:** (empty)
- * Short description:** (empty)
- Description:** (empty)

Related Search Section:

- Related Search:** (empty search bar)
- Knowledge & Catalog (All):** (dropdown menu)
- Results:** No results to display

18. LIST EDIT BLOCKING TEST

Test Case 4 – Test List Edit Blocking

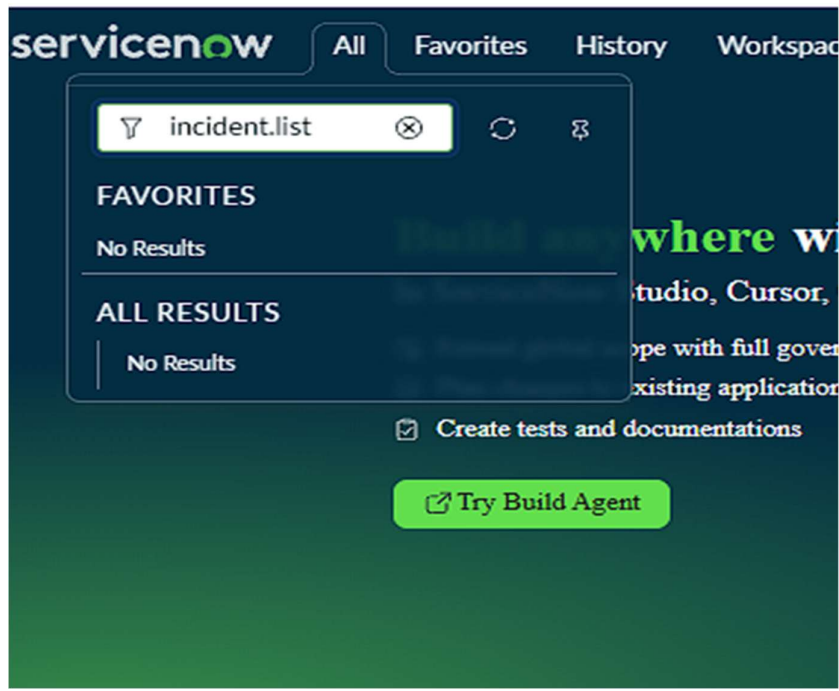
Steps:

1. Navigate to Incident → All.
2. Locate an Incident.
3. Double-click the State field.
4. Try to change the State.

Expected Result

- An alert message should appear.
- The State value should remain unchanged.
- The direct list edit should be blocked.

This confirms that the onCellEdit Client Script is working correctly.



servicenow All Favorites History Workspaces Admin Incident - Create INC0010006 Search

Incident New record Submit Resolve

Urgency set to High for High impact incident.

Number	INC0010006	Channel	-- None --
* Caller	Abel Tutor	State	Resolved
Category	Inquiry / Help	Impact	1 - High
Subcategory	-- None --	Urgency	1 - High
Service		Priority	1 - Critical
Service offering		Assignment group	CAB Approval
Configuration item		Assigned to	Bernard Laboy
* Short description			
Description			

Related Search Results

Related Search Knowledge & Catalog (All)

No results to display

19. TEST CASE 5 – FORM-BASED STATE UPDATE

Steps

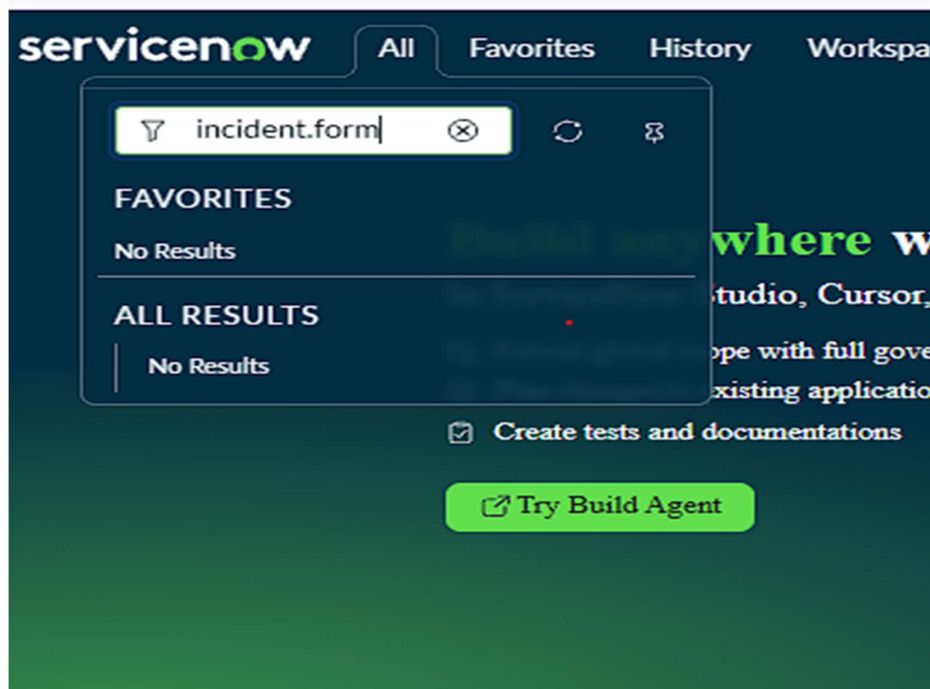
1. Open an Incident record.
2. Change the State field from the Incident form.
3. Click Update.

Expected Result

The State change should be saved successfully.

This confirms that the State restriction is specifically applied to direct list editing while changes made through the Incident form are allowed.

The other configured rules, including Assigned To and Urgency behavior, should continue to work according to the defined conditions.



dev446133.service-now.com/now/nav/ui/classic/params/target/incident.do%3Fsys_id%3D479f2c64476f031023189d74116d4363%26sysparm_record_targ... Verify it's you

servicenow All Favorites History Workspaces Admin Incident - INC0010002 Search Discuss Follow Update Resolve Delete Responses

Incident menu: Save, Add to Visual Task Board, Copy Incident, Create Incident Task, Create Outage, Create Request, Create Child Incident, Refresh Impacted Services, Metrics Timeline, Follow on Live Feed, Analyze Access, Configure, Export, View, Create Favorite, Copy URL, Copy sys_id, Show XML, History, Reload form

Incident details: Incident: INC0010002, Channel: -- None --, State: Resolved, Impact: 1 - High, Urgency: 1 - High, Priority: 1 - Critical, Assignment group: CAB Approval, Assigned to: Bernard Laboy

Related Search Results: Related Search: hii, Knowledge & Catalog (All), No matching results found for hii

dev446133.service-now.com/now/nav/ui/classic/params/target/incident_list.do%3Factive%3Dtrue%26sysparm_query%3Dactive%253Dtrue%255EEQ Verify it's you

servicenow All Favorites History Workspaces Admin Incidents Search Actions on selected rows... New

All > Active = true

	Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
☐	INC0010003	2026-09-21 23:02:47	Hii	Abel Tuter	1 - Critical	In Progress	Inquiry / Help	CAB Approval	Bernard Laboy	2026-09-24 23:05:21	system
☒	INC0010002	2026-09-21 22:37:02	practise	Abel Tuter	4 - Low	New	Inquiry / Help	(empty)	(empty)	2026-09-21 22:37:46	admin
☐	INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:30:24	admin
☐	INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
☐	INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
☐	INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
☐	INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-22 14:05:48	admin
☐	INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
☐	INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
☐	INC0007001	2018-10-16 22:47:10	Employee payroll application server is down.	David Miller	1 - Critical	New	Hardware	Openspace	(empty)	2026-06-12 15:15:11	system

1 to 20 of 42

dev446133.service-now.com/now/nav/ui/classic/params/target/incident_list.do%3Factive%3Dtrue%26sysparm_query%3Dactive%253Dtrue%255EEQ

dev446133.service-now.com says
State cannot be updated using list editing.Please open the incident.

OK

Incidents

Number

Search

All > Active = true

Number	Opened	Short description	Assignment group	Assigned to	Updated	Update				
INC0010003	2026-09-21 23:02:47	Hii	Abel Tutor	Critical	Progress	Inquiry / Help	Case Approval	Bernard Laboy	2026-09-24 23:05:21	system
INC0010002	2026-09-21 22:37:02	practise	Abel Tutor	4 - Low	Resolved		(empty)	(empty)	2026-09-21 22:37:46	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:30:24	admin
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-22 14:05:48	admin
INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
INC0007001	2018-10-16 22:47:10	Employee payroll application server is down.	David Miller	1 - Critical	New	Hardware	Openspace	(empty)	2026-06-12 15:15:11	system

1 to 20 of 42

20. EXPECTED RESULTS

After successful implementation, the following results are expected:

1. High Impact Incidents trigger the High Impact Control UI Policy.
2. Assignment Group becomes mandatory when Impact is High.
3. Urgency becomes read-only when Impact is High.
4. Urgency is automatically set to High when Impact changes to High.
5. A High Impact Incident cannot be saved when Assigned To is empty.
6. An appropriate error message is displayed during invalid submission.
7. Direct State changes through list editing are prevented.
8. State changes through the Incident form are allowed.
9. UI Policy changes are reversed when Impact is changed from High to another value.
10. Incident records contain more consistent and valid information.

21. ADVANTAGES

21.1 Improved Data Quality

The project reduces incomplete and incorrect Incident information by applying defined rules.

21.2 Automated Field Updates

Urgency can be automatically set based on the Impact value.

21.3 Better User Interaction

Users receive immediate feedback when an action does not satisfy the defined conditions.

21.4 Consistent Incident Records

The configured rules ensure that similar Incident conditions receive consistent field behavior.

21.5 Controlled List Editing

The State field cannot be directly changed through list editing.

21.6 Improved Validation

The onSubmit Client Script ensures that required information is available before saving under the specified condition.

21.7 Lightweight Implementation

The solution uses ServiceNow client-side controls and is suitable for demonstrating practical configuration skills in a short-duration project.

The source document describes the solution as lightweight, efficient, and suitable for a short-duration micro project.

22. LIMITATIONS

The project focuses on client-side controls for the ServiceNow Incident form.

The current implementation has the following limitations:

1. The project is limited to the Incident table.
2. The implemented validations are focused on the specified Incident conditions.
3. The project demonstrates client-side behavior rather than a complete enterprise Incident Management implementation.
4. Advanced server-side validation is not included in the defined implementation.
5. External system integrations are not included.
6. Advanced workflow automation is outside the defined project scope.

These limitations represent the scope of the current project implementation.

23. FUTURE ENHANCEMENTS

The project can be extended in the future with additional ServiceNow features.

Possible Enhancements

1. Add additional UI Policies for different Incident conditions.
2. Add more Client Scripts for advanced validation.
3. Implement additional automatic field population.
4. Add server-side validation for stronger data enforcement.
5. Implement notification mechanisms for critical Incidents.
6. Add automated assignment based on Incident information.
7. Create dashboards and reports for Incident analysis.
8. Add more advanced Incident workflow automation.
9. Extend validation rules to additional Incident fields.
10. Integrate the Incident process with other ServiceNow modules.

These enhancements can make the system more comprehensive while keeping the existing client-side controls as the foundation.

24. CONCLUSION

The “**Implement Client Script & UI Policy (Incident)**” project successfully demonstrates how ServiceNow UI Policies and Client Scripts can be used together to improve Incident data quality and user interaction.

A **High Impact Control** UI Policy was created to apply additional controls when an Incident has High Impact. UI Policy Actions were used to make Assignment Group mandatory and to control the Urgency field.

An **onChange Client Script** was implemented to automatically set Urgency to High when Impact is High. An **onSubmit Client Script** was implemented to prevent the Incident from being saved when Assigned To is missing under the specified condition. An **onCellEdit Client Script** was also implemented to prevent direct State changes through list editing.

The project was tested through mandatory validation, successful save, reverse condition, list-edit blocking, and form-based update scenarios.

Overall, the project demonstrates practical use of **Incident Management, UI Policies, UI Policy Actions, Client Scripts, and Form Validation** to enforce defined business rules and maintain clean and consistent Incident data.

The source project concludes that combining onChange, onSubmit, and onCellEdit Client Scripts with UI Policy Actions provides an effective way to control dynamic field behavior and data submission in ServiceNow.